



Roadmap

Phase 1: Hybrid Reactivity Core

(3-4 weeks |   )

Milestone	Icon	Key Points	Portfolio Tip
Core Signals		- <code>signal()</code> / <code>computed()</code> / <code>effect()</code> API- Explicit dependency tracking- Microtask batching	Write blog: "Why Signals > Observables"
Deep Reactivity Layer		- Opt-in <code>deepSignal()</code> - Proxy-based nesting- Memory-safe with WeakMap	Demo: <i>Signal vs Proxy performance test</i>
Batching & Cleanup		- <code>batch()</code> for atomic updates- Context-based destruction- Zombie effect prevention	Video: <i>Effect execution timeline</i>
Debugging Tools		- Dev-mode dependency warnings- Signal graph visualization	Build: <i>Interactive dependency viewer</i>

Timeline

Copy

Week 1:  Core Signals
Week 2:  Deep Reactivity
Week 3:  Batching & Cleanup
Week 4:  Debugging Tools

Key Deliverables Preview

1. Code Example:

ts

Copy

```
const [count, set] = signal(0);
const user = deepSignal({ profile: { name: "Alice" } });
effect(() => console.log(count(), user.profile.name));
```

2. Portfolio Hook:

- "Hybrid Reactivity: When to Use Signals vs Proxies" guide
- Benchmark vs Angular/Vue (3x faster updates 🚀)

Design Principles

- 🛠️ **Opt-In Complexity:** Start simple (Angular-style), add magic later
- 📄 **Memory-Safe:** WeakMap + GC-friendly patterns
- 🔄 **Framework-Agnostic:** Isolate reactivity for future reuse

Key Takeaway: This phase builds a reactivity system that's *explicit like Angular*, *deep like Vue*, and *debuggable like Solid*.

Phase 2: Compiled Template Layer

(3-4 weeks | 🛠️📄⚡)

Milestone	Icon	Key Points	Portfolio Tip
Template Compiler	🛠️	- HTML → Optimized JS-Signal-aware output- Partial hydration directives	Blog: "Compiler vs Runtime Reactivity"
DOM Optimization	⚡	- Skip VDOM for direct updates- Keyed list reconciliation	Demo: 10k items benchmark vs React
Hydration Engine	💧	- SSR support- Island state serialization- Client hydration guards	Case study: SEO-friendly SPA

Phase 3: Component & Island Architecture

(4-5 weeks | 🌴📦🔗)

Milestone	Icon	Key Points	Portfolio Tip
Component Model	📦	- Lifecycle hooks- Isolated signal state- Props/events system	Build: <i>Angular-like component demo</i>
Island System	🌴	- Partial hydration- Intersection-based loading- Lazy signal init	Video: <i>Hydration waterfall visualization</i>
Cross-Island Comm	📡	- Custom events- Shared context API- Portal support	Guide: <i>"Microfrontends with Islands"</i>

Phase 4: Advanced Features & Optimization

(4-6 weeks | 🚀📊🧩)

Milestone	Icon	Key Points	Portfolio Tip
DevTools Integration	🔧	- Signal dependency graph- Component tree inspector- Performance profiler	Open-source: <i>Browser extension</i>
Optimization Suite	🧩	- Lazy loading- Signal memoization- Tree- shakeable runtime	Benchmark: <i>Bundle size vs Svelte</i>
SSR/SSG Support	🌐	- Full SSR pipeline- Static site generation- Hybrid rendering modes	Case study: <i>Blog migration journey</i>

Phase 5: Ecosystem & Polish

(3-5 weeks | 🧰🧪📖)

Milestone	Icon	Key Points	Portfolio Tip
CLI & Tooling	⚙️	- Project scaffolding- Build optimizations- AOT compilation	Release: <i>Starter kit on npm</i>

Plugin System		- Custom directives- Signal middleware- Compiler hooks	Showcase: <i>State management plugin</i>
Documentation		- Interactive tutorials- API reference- Migration guides	Publish: <i>"Framework Philosophy" doc</i>

Complete Timeline

Copy

Phase 1 (4w) :     // Reactivity Core
Phase 2 (4w) :    // Templates & DOM
Phase 3 (5w) :    // Components & Islands
Phase 4 (6w) :    // Advanced Features
Phase 5 (5w) :    // Ecosystem Polish

Key Deliverables Preview

1. Code Example:Run HTML

html

Copy

```
<!-- Component template →
<Counter client:load><script>
  const [count, set] = signal(0);
</script>
<button @click={() ⇒ set(count() + 1)}>
  Clicks: {{ count }}
</button></Counter>
```

2. Portfolio Hooks:

- Interactive tutorial: *"Build a Twitter Clone in 15m"*
- Architecture diagram: *Signal flow in Island apps*

- Performance case study: *From React to Custom Framework*
-

Design Principles

- 🎯 **Gradual Adoption:** Start with simple scripts → scale to full apps
- 📦 **Modular Architecture:** Use standalone reactivity/compiler
- 🔗 **Web Standards First:** Custom elements, ES modules, Shadow DOM